Experiment no. 03

Aim: To deploy multi-service application using Docker compose

LO4: To automate provisioning, configuration, container orchestration, and cloud resource management using Ansible, Docker, Kubernetes, and Terraform.

Theory:

Docker Compose – Simple Points

- **Docker Compose** is a tool provided by Docker for running **multiple containers together**.
- It uses a **YAML configuration file**, usually named docker-compose.yml.
- Instead of creating containers one by one using separate docker run commands, we can define **all services in one file**.
- A multi-container application may contain services such as:
    - Web server o Database
    - Cache o Other supporting services
- Docker Compose allows us to manage all these services **together as one application**.
- The YAML file can define:
    - Container **images** o **Environment variables** o **Ports** o **Volumes**
    - **Dependencies**
- The command:

```
docker compose up
```

reads the YAML file and **creates and starts all the required containers**.

- Docker Compose automatically creates a **common network** for the services.
- Containers can communicate with each other using their **service names as hostnames**.
- We can easily **start, stop, and manage** all containers together.
- It reduces manual work and makes container management **easier, faster, and less errorprone**.

- **Main purpose:** To deploy and manage a **multi-service application as a single unit** instead of managing each container separately.

Key concepts used in this experiment:

- Service: A single container definition within the compose file (e.g. web, database, phpmyadmin). Each service specifies the image to use, ports to expose, environment variables, and other configuration.
- Image: The base image each service is built from — nginx for the web server, mysql:8.0 for the database, and phpmyadmin/phpmyadmin for the database admin UI.
- Ports mapping ("host:container"): Maps a port on the host machine to a port inside the container, e.g. "8080:80" exposes the container's port 80 on the host's port 8080.
- Environment variables: Used to configure a service at startup without modifying the image itself — here they set the MySQL root password, database name, and user credentials, and tell phpMyAdmin which database host to connect to.
- Volumes: Named, persistent storage (mysql-data) that keeps database data intact even if the container is stopped, removed, or recreated.
- depends_on: Specifies that one service depends on another, influencing the order in which
- containers are started (phpmyadmin is started after database).
- This experiment uses three services instead of the minimum two, to demonstrate a more realistic multi-service setup: an nginx web server, a MySQL database, and a phpMyAdmin instance for managing that database through a browser-based interface — showing how Compose handles inter-service communication (phpMyAdmin connecting to the database service purely by its service name, "database").

DOCKER COMPOSE FILE (docker-compose.yml)

The following configuration was created inside the compose-demo directory and saved as dockercompose.yml:

```
version: '3.8'  services:  web:
image: nginx  container_name: webserver
ports:  - "8090:80"  database:  image:
mysql:8.0  container_name: mysqlserver
environment:
  MYSQL_ROOT_PASSWORD: root123
```

MYSQL_DATABASE: studentdb

MYSQL_USER: student

MYSQL_PASSWORD: student123 ports:

- "3307:3306" volumes: - mysql-data:/var/lib/mysql phpmyadmin:

image: phpmyadmin/phpmyadmin

container_name: phpmyadmin

environment: PMA_HOST: database

PMA_USER: student PMA_PASSWORD: student123 ports: - "8081:80"

depends_on:

- database

volumes: mysql- data:

PROCEDURE

The following steps were carried out to set up and deploy the application:

	Step / Command	Description
1	mkdir composedemo	Creates a new project directory for this experiment.
2	cd compose-demo	Moves into the newly created directory.
3	(create dockercompose.yml)	The compose file is created using Notepad (saved as "All Files" so it is not saved with a .txt extension) inside the compose-demo folder.
4	dir	Lists the contents of the folder to confirm dockercompose.yml was saved correctly.
5	docker compose up -d	Reads docker-compose.yml and creates + starts all three services (webserver, mysqlserver, phpmyadmin) in detached (background) mode.

How the services connect

The web service (nginx) is accessible from the host browser at http://localhost:8090.

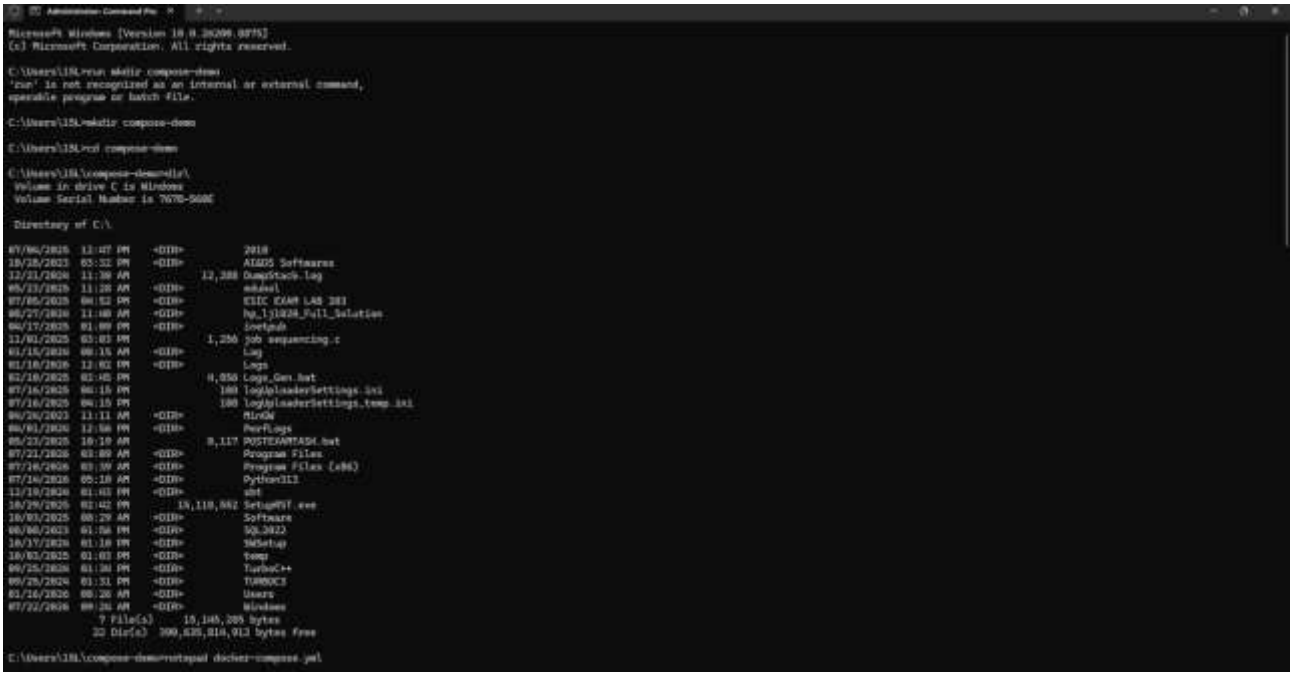
The database service (MySQL) is accessible on host port 3307, with credentials and database name set via environment variables, and its data persisted in the mysql-data volume.

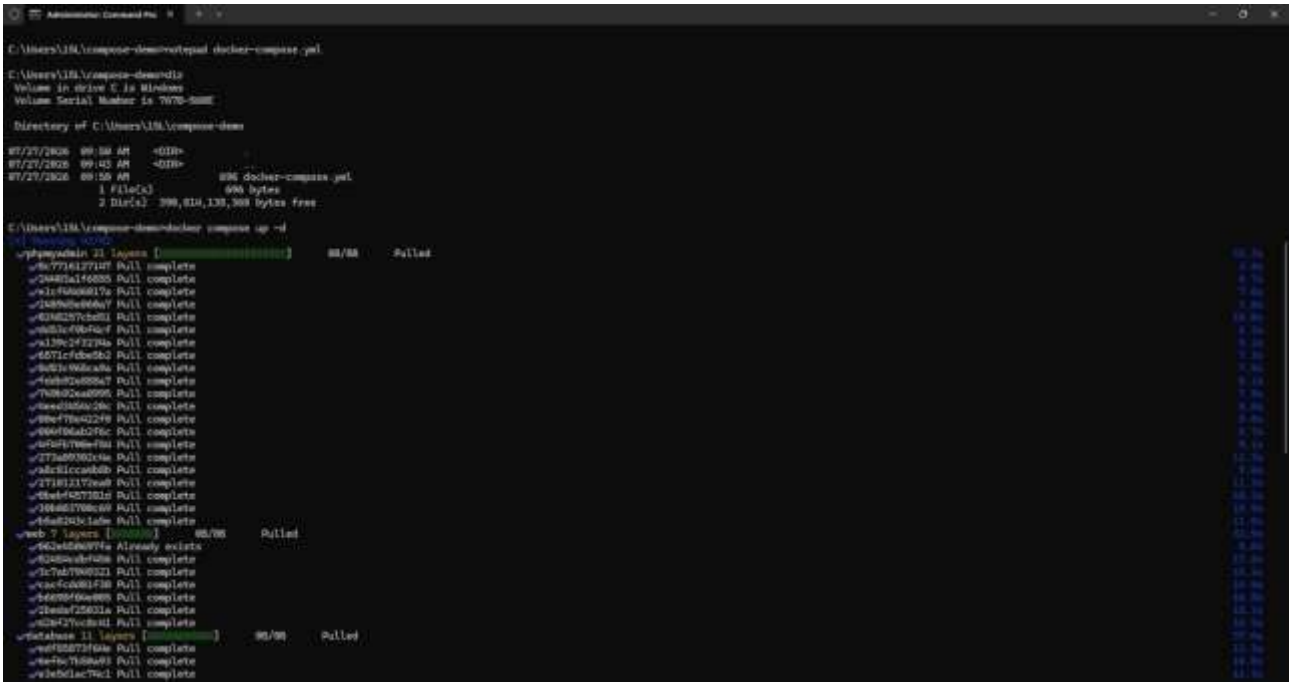
The phpmyadmin service is accessible at http://localhost:8081, and connects internally to the database service using the hostname "database" (its Compose service name) rather than an IP address — this works because Compose automatically places all services on the same internal network.

Note

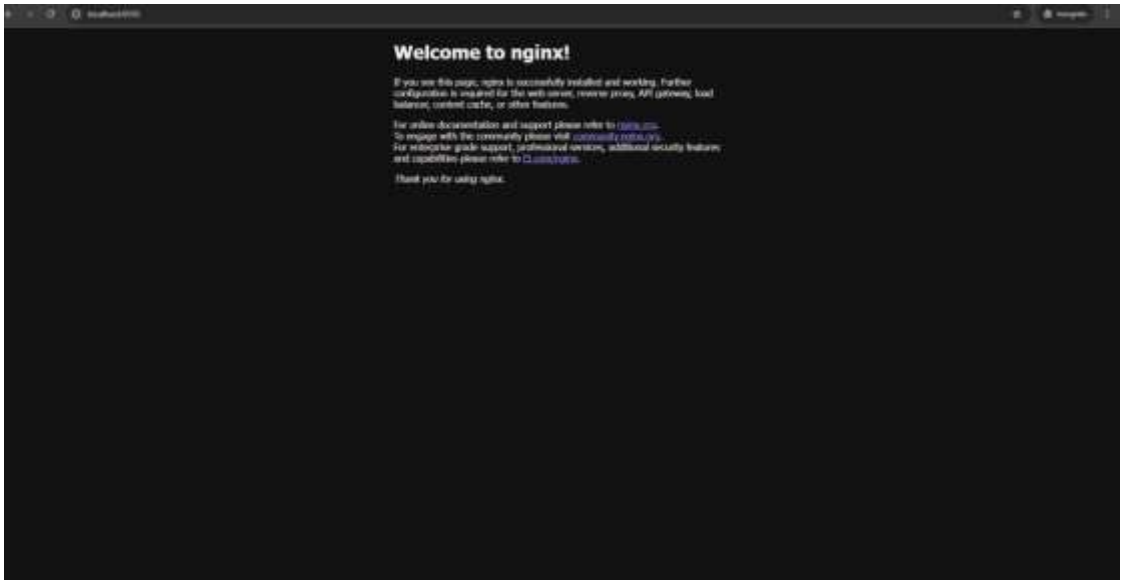
During deployment, docker compose up -d initially failed because the default host ports (3306 and 8080) specified in the file were already in use on the host machine. This was resolved by changing only the host-side port numbers to 3307 and 8090 respectively in docker-compose.yml, without altering the container-side ports, after which docker compose down followed by docker compose up -d started all three services successfully.

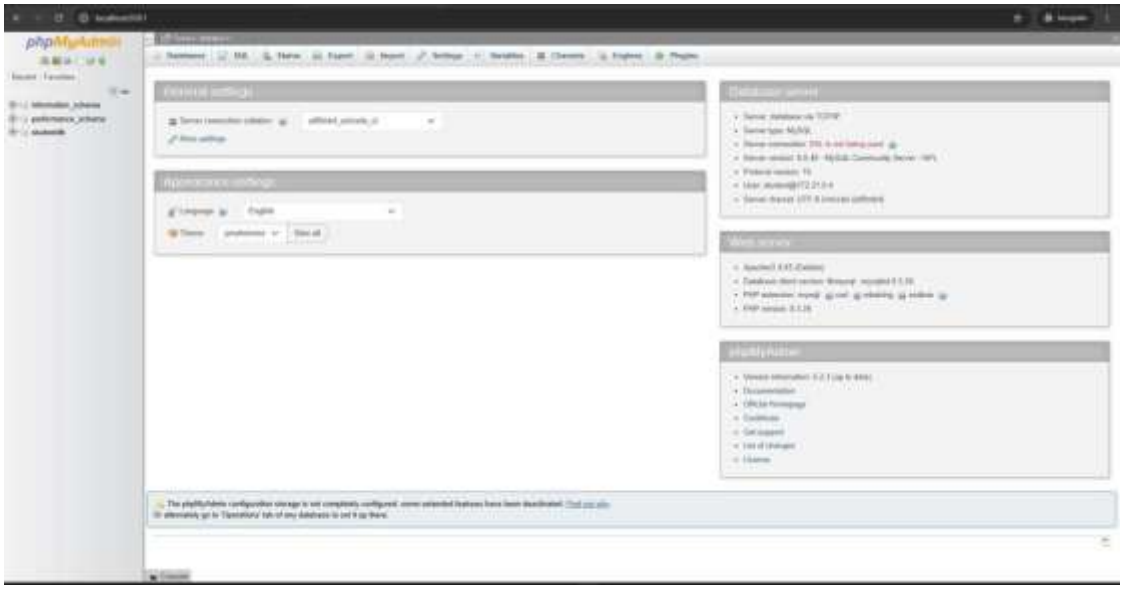
Observations:











DOCKER COMPOSE COMMANDS USED

The following Docker Compose commands were used to manage the lifecycle of the multiservice application:

#	Command	Description
1	docker compose up -d	Starts all services defined in the compose file, in detached mode.
2	docker compose ps	Shows the status of all running services defined in the compose file.

3	docker compose logs	Displays combined logs from all services, useful for debugging.
4	docker compose images	Lists the images used by each service in the compose file.
5	docker compose config	Validates the syntax of the compose file and displays the fully resolved configuration.
6	docker compose stop	Stops all running services without removing the containers.
7	docker compose start	Starts services that were previously stopped.
8	docker compose restart	Restarts all containers defined in the compose file.
9	docker compose down	Stops and removes all containers and the default network, but keeps named volumes.
10	docker compose down -v	Stops and removes all containers, networks, AND named volumes (deletes database data) — run this last.

Conclusion

- A **multi-service application** with a web server, database, and database administration tool was successfully deployed using **Docker Compose**.
- All services were defined in a single docker-compose.yml file.
- The docker compose up -d command started all containers together.
- docker compose ps confirmed that all containers were running and properly connected.
- Docker Compose makes **multi-container deployment easier, organized, reproducible, and manageable**.